

Construção de Software

Encontro 3

Encapsulamento e abstração

Objetivos do Encontro

- Apresentar conceitos centrais sobre **Encapsulamento e abstração**.
- Discutir exemplos práticos com UML e código.
- Fixar conteúdos com exercícios práticos.

Introdução

Nesta aula abordaremos:

- Definições fundamentais
- Benefícios e aplicações
- Boas práticas

Encapsulamento

Encapsulamento é um dos pilares fundamentais da programação orientada a objetos que consiste em:

- **Ocultar os detalhes internos** de implementação de uma classe
- **Agrupar dados e métodos** relacionados em uma única unidade (classe)
- **Controlar o acesso** aos atributos através de métodos públicos (getters/setters)
- **Proteger a integridade** dos dados através de validações

Benefícios:

- Manutenibilidade do código
- Reutilização de componentes
- Redução de acoplamento
- Maior segurança dos dados

Exemplo prático – Encapsulamento

```
public class ContaBancaria {  
    private double saldo; // Atributo privado - encapsulado  
    private String numeroConta;  
  
    public ContaBancaria(String numeroConta, double saldoInicial) {  
        this.numeroConta = numeroConta;  
        this.saldo = saldoInicial;  
    }  
  
    // Métodos públicos para acesso controlado  
    public double getSaldo() {  
        return saldo;  
    }  
  
    public void depositar(double valor) {  
        if (valor > 0) {  
            saldo += valor;  
        }  
    }  
  
    public boolean sacar(double valor) {  
        if (valor > 0 && valor <= saldo) {  
            saldo -= valor;  
            return true;  
        }  
        return false;  
    }  
}
```

Ocultação de informações

Ocultação de informações (Information Hiding) é um princípio que estabelece:

- **Esconder detalhes de implementação** que não são relevantes para o usuário da classe
- **Expor apenas interfaces necessárias** para interação com o objeto
- **Proteger dados sensíveis** evitando acesso direto
- **Facilitar mudanças internas** sem afetar código cliente

Características:

- Uso de modificadores de acesso (private, protected, public)
- Métodos de acesso controlado
- Validação de dados na entrada
- Abstração de complexidade interna

Exemplo prático – Ocultação de informações

```
public class Funcionario {
    private String nome;
    private double salario; // Informação sensível - oculta
    private String cpf;     // Informação sensível - oculta

    public Funcionario(String nome, double salario, String cpf) {
        this.nome = nome;
        this.salario = salario;
        this.cpf = cpf;
    }

    // Acesso público apenas ao necessário
    public String getNome() {
        return nome;
    }

    // Método que não expõe informação sensível diretamente
    public boolean temSalarioSuperiorA(double valor) {
        return salario > valor;
    }

    // CPF mascarado para proteção
    public String getCpfMascarado() {
        return "***." + cpf.substring(3, 6) + ".***-***";
    }
}
```

Abstração de dados

Abstração de dados é o processo de simplificação que:

- **Foca nos aspectos essenciais** de um objeto, ignorando detalhes irrelevantes
- **Define tipos de dados abstratos** que especificam operações sem revelar implementação
- **Estabelece contratos** através de interfaces e classes abstratas
- **Separa "o que fazer" do "como fazer"**

Elementos principais:

- Classes abstratas e interfaces
- Métodos abstratos que definem comportamento
- Implementações concretas específicas
- Polimorfismo através de abstrações

Exemplo prático – Abstração de dados

```
// Abstração de uma estrutura de dados Lista
public abstract class Lista<T> {
    protected int tamanho;

    public abstract void adicionar(T elemento);
    public abstract T remover(int indice);
    public abstract T obter(int indice);

    // Método concreto comum a todas as implementações
    public int getTamanho() {
        return tamanho;
    }

    public boolean estaVazia() {
        return tamanho == 0;
    }
}
```

Exemplo prático – Implementação de abstração

```
// Implementação concreta
public class ListaArray<T> extends Lista<T> {
    private T[] elementos;
    private static final int CAPACIDADE_INICIAL = 10;

    public ListaArray() {
        elementos = (T[]) new Object[CAPACIDADE_INICIAL];
        tamanho = 0;
    }

    @Override
    public void adicionar(T elemento) {
        elementos[tamanho++] = elemento;
    }

    @Override
    public T obter(int indice) {
        return elementos[indice];
    }

    @Override
    public T remover(int indice) {
        T elemento = elementos[indice];
        // Lógica de remoção...
        tamanho--;
        return elemento;
    }
}
```

Exercício – Abstração de dados

Proposta de exercício para aplicar o conceito de **Abstração de dados**.

Interfaces de acesso

Interfaces de acesso são contratos que definem:

- **Métodos que devem ser implementados** por classes concretas
- **Padronização de comportamentos** entre diferentes implementações
- **Desacoplamento** entre código cliente e implementação específica
- **Flexibilidade** para trocar implementações sem afetar o sistema

Vantagens:

- Facilita testes unitários (mocking)
- Permite múltiplas implementações
- Reduz dependências diretas
- Melhora a manutenibilidade do código

Exemplo prático – Interfaces de acesso

```
// Interface define o contrato de acesso
public interface RepositorioUsuario {
    void salvar(Usuario usuario);
    Usuario buscarPorId(Long id);
    List<Usuario> listarTodos();
    void deletar(Long id);
}
```



```
public class RepositorioUsuarioBD implements RepositorioUsuario {
    private ConexaoBanco conexao;

    @Override
    public void salvar(Usuario usuario) {
        // Implementação específica para BD
        String sql = "INSERT INTO usuario (nome, email) VALUES (?, ?)";
        // ... código de inserção
    }

    @Override
    public Usuario buscarPorId(Long id) {
        // Implementação específica para BD
        return null; // Simplificado
    }

    @Override
    public List<Usuario> listarTodos() {
        // Implementação específica para BD
        return new ArrayList<>(); // Simplificado
    }

    @Override
    public void deletar(Long id) {
        // Implementação específica para BD
    }
}
```

```
// Implementação para arquivo
public class RepositorioUsuarioArquivo implements RepositorioUsuario {
    private String caminhoDiretorio;

    @Override
    public void salvar(Usuario usuario) {
        String caminhoArquivo = caminhoDiretorio + "/" + usuario.getId() + ".txt";
        File arquivo = new File(caminhoArquivo);
        //Serializar e salvar o usuário no arquivo
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(arquivo))) {
            oos.writeObject(usuario);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    @Override
    public Usuario buscarPorId(Long id) {
        String caminhoArquivo = caminhoDiretorio + "/" + id + ".txt";
        File arquivo = new File(caminhoArquivo);
        if (!arquivo.exists()) {
            return null;
        }
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(arquivo))) {
            return (Usuario) ois.readObject();
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

Boas práticas de encapsulamento

Boas práticas de encapsulamento incluem diretrizes para:

- **Sempre tornar atributos privados** e fornecer acesso através de métodos
- **Validar dados** nos setters antes de atribuir valores
- **Implementar métodos comportamentais** que façam sentido para a classe
- **Evitar getters/setters desnecessários** - expor apenas o que for necessário
- **Usar construtores com validação** para garantir estado válido desde a criação
- **Manter a coesão** - cada classe deve ter uma única responsabilidade

Exemplo prático – Boas práticas de encapsulamento

```
public class Produto {
    // 1. Atributos privados
    private String nome;
    private double preco;
    private int quantidade;
    private LocalDate dataVencimento;

    // 2. Construtor com validação
    public Produto(String nome, double preco, int quantidade, LocalDate dataVencimento) {
        setNome(nome);
        setPreco(preco);
        setQuantidade(quantidade);
        setDataVencimento(dataVencimento);
    }

    // 3. Getters simples
    public String getNome() { return nome; }
    public double getPreco() { return preco; }
    public int getQuantidade() { return quantidade; }

    // 4. Setters com validação
    public void setNome(String nome) {
        if (nome == null || nome.trim().isEmpty()) {
            throw new IllegalArgumentException("Nome não pode ser vazio");
        }
        this.nome = nome.trim();
    }

    public void setPreco(double preco) {
        if (preco < 0) {
            throw new IllegalArgumentException("Preço não pode ser negativo");
        }
        this.preco = preco;
    }
}
```

```
    public void setQuantidade(int quantidade) {
        if (quantidade < 0) {
            throw new IllegalArgumentException("Quantidade não pode ser negativa");
        }
        this.quantidade = quantidade;
    }

    public void setDataVencimento(LocalDate dataVencimento) {
        if (dataVencimento != null && dataVencimento.isBefore(LocalDate.now())) {
            throw new IllegalArgumentException("Data de vencimento não pode ser no passado");
        }
        this.dataVencimento = dataVencimento;
    }

    // 5. Métodos comportamentais
    public boolean estaVencido() {
        return dataVencimento != null && dataVencimento.isBefore(LocalDate.now());
    }

    public double calcularValorTotal() {
        return preco * quantidade;
    }

    public void adicionarEstoque(int quantidade) {
        if (quantidade <= 0) {
            throw new IllegalArgumentException("Quantidade deve ser positiva");
        }
        this.quantidade += quantidade;
    }

    public boolean removerEstoque(int quantidade) {
        if (quantidade <= 0 || quantidade > this.quantidade) {
            return false;
        }
        this.quantidade -= quantidade;
        return true;
    }
}
```

Padrão de projeto: DAO (Data Access Object)

Conceito

O padrão DAO é uma abordagem para separar a lógica de acesso a dados da lógica de negócios de uma aplicação. Ele fornece uma interface para realizar operações de CRUD (Create, Read, Update, Delete) em objetos de domínio, permitindo que a aplicação interaja com diferentes fontes de dados (bancos de dados, serviços web, etc.) de forma transparente.

Implementação

A implementação do padrão DAO geralmente envolve a criação de uma interface que define os métodos de acesso a dados e uma ou mais classes concretas que implementam essa interface. As classes DAO são responsáveis por interagir com a fonte de dados, como um banco de dados relacional ou um serviço web.

```
public interface UsuarioDAO {  
    void salvar(Usuario usuario);  
    Usuario buscarPorId(Long id);  
    List<Usuario> listarTodos();  
    void deletar(Long id);  
}
```

```

public class UsuarioDAOImpl implements UsuarioDAO {
    private ConexaoBanco conexao;

    public UsuarioDAOImpl(ConexaoBanco conexao) {
        this.conexao = conexao;
    }

    @Override
    public void salvar(Usuario usuario) {
        // LOGICA PARA SALVAR O USUÁRIO NO BANCO DE DADOS
    }

    @Override
    public Usuario buscarPorId(Long id) {
        // LOGICA PARA BUSCAR O USUÁRIO POR ID NO BANCO DE DADOS
        return null;
    }

    @Override
    public List<Usuario> listarTodos() {
        // LOGICA PARA LISTAR TODOS OS USUÁRIOS NO BANCO DE DADOS
        return null;
    }

    @Override
    public void deletar(Long id) {
        // LOGICA PARA DELETAR O USUÁRIO NO BANCO DE DADOS
    }
}

```

Padrão de Projeto: Façade

Conceito

O padrão Façade é uma abordagem para fornecer uma interface simplificada e unificada para um conjunto de interfaces em um subsistema. Ele oculta a complexidade do sistema e facilita a interação com ele, permitindo que os clientes acessem funcionalidades de forma mais fácil e intuitiva.

Implementação

A implementação do padrão Façade envolve a criação de uma classe que encapsula as interações com várias classes ou subsistemas, fornecendo métodos simplificados para os clientes.

```
public class SistemaPagamentoFacade {  
    private ProcessadorCartaoCredito processadorCartao;  
    private ServicoPagamentoOnline servicoPagamento;  
    private GerenciadorFaturas gerenciadorFaturas;  
  
    public SistemaPagamentoFacade() {  
        this.processadorCartao = new ProcessadorCartaoCredito();  
        this.servicoPagamento = new ServicoPagamentoOnline();  
        this.gerenciadorFaturas = new GerenciadorFaturas();  
    }  
    // Métodos simplificados para os clientes  
    public boolean processarPagamentoCartao(String numeroCartao, double valor) {  
        if (processadorCartao.validarCartao(numeroCartao)) {  
            return processadorCartao.efetuarPagamento(numeroCartao, valor);  
        }  
        return false;  
    }  
    // ...  
}
```

Exercício – Boas práticas de encapsulamento

Você precisará criar um cadastro de produtos para um sistema de inventário. Para isso, implemente a classe `Produto` seguindo as boas práticas de encapsulamento discutidas:

- Atributos privados: nome, custo unitário, quantidade.
- Construtor que inicializa todos os atributos com validação.
- Getters para todos os atributos, setter para nome.
- Um método `adicionarItem` que recebe valor (quantidade e valor).
- Um método específico para remover estoque (quantidade).
- O custo unitário deve ser calculado conforme a média ponderada dos itens em estoque.
- O preço deve ser 10% maior que o custo de unidade.

Referências

- Page-Jones, M. *Fundamentos do Desenho Orientado a Objeto com UML*. Makron Books.
- Larman, C. *Utilizando UML e Padrões*. Saraiva.
- Ementa da disciplina.